FMOD Engine User Manual 2.03

1. Welcome to the FMOD Engine

2. Core API Key Concepts

- 2.1 The Core API and the FMOD Engine
- 2.2 The System Object
- 2.3 Sounds
- 2.4 Channels
  - 2.4.1 Channel Virtualization
- 2.5 Channel Groups
- 2.6 DSP Units
- 2.7 FMOD Sound Banks
- 2.8 3D Sounds and Channels
- 2.9 Streams
- 2.10 Audio Devices

3. Core API Getting Started

4. Core API Loading and Playing Sounds

5. Core API Spatializing Sounds

6. Core API Mixing and Routing

7. Core API Using DSP Effects

8. Core API Effect Reference

9. Core API Managing Resources

10. Core API Advanced Topics

11. Core API Reference

12. Studio API Getting Started

13. Studio API Guide

14. Studio API 3D Events

15. Studio API Threads

16. Studio API Reference

17. Platform Details

18. Plug-in DSP API Guide

19. Plug-in API Reference

20. FSBank API Reference

21. Troubleshooting

22. Glossary

2. Core API Key Concepts

The Core API is a programmer API that allows the manipulation of low-level audio primitives, and is the backbone of the FMOD Engine.

While the Studio API is easy to use and well-suited to most game projects, the Core API is both more powerful and more flexible. This makes it useful for games with unusual and strict audio requirements that do not perfectly fit into the Studio API's paradigm of events and buses. In addition, even games made using the Studio API can benefit from a solid knowledge of the Core API, as the Core API is the foundation upon which the Studio API is built.

This chapter introduces several essential FMOD Core API concepts. Understanding these concepts and how they interact is key to understanding how best to use the Core API to develop adaptive game audio.

This chapter is designed to be read before proceeding on to the rest of this manual, as the concepts introduced here come up frequently in later chapters.

2.1 The Core API and the FMOD Engine

The FMOD Engine is a runtime library for playing adaptive audio in games. It consists of two APIs: FMOD Studio API, and the FMOD Core API.

The FMOD Core API allows audio programmers to create audio content without using FMOD Studio, and to interact with the FMOD Engine's underlying mechanisms. This makes it more powerful and flexible than the FMOD Studio API.

The FMOD Studio API can load and play .bank files created in FMOD Studio, an application that allows sound designers and composers to create adaptive audio content for games. This makes it less flexible than the FMOD Core API, but easier to use, especially for sound designers with limited audio programming experience and audio programmers with limited experience of sound design.

FMOD for Unity and FMOD for Unreal Engine are packaged software integrations of the FMOD Engine for use in Unity and Unreal Engine games. Each package includes a copy of the FMOD Engine that is automatically installed along with the integration.

FMOD version numbers are split into three parts, in the format: *productVersion.majorVersion.minorVersion.*

2.2 The System Object

The FMOD system object is the heart of the FMOD Engine, the key mechanism that all other Core API features depend on to work. This means that before your game can do anything else with the Core API, the system object needs to be created. Outside of exceptional circumstances, you only ever need one FMOD system object to handle all the audio in your game. Most games therefore create a system object when the game is launched, and then only destroy that system object when the player quits the game.

For information about getting your game ready to play audio with the FMOD Core API, see the Running the Core API chapter. For more information about the system object, see System subchapter of the Core API Reference chapter.

2.3 Sounds

A sound is a piece of sample data that's loaded or buffered into memory, ready to be played. Sounds are created using System::createStream or System::createSound, which also lets you choose the loading mode to be used for that sound.

Once a sound is loaded, you can play it with System::playSound. This creates a channel that plays the sound's sample data.

For more information about loading and playing sounds, see the Loading and Playing Sounds in the Core API chapter. For reference information about sounds, see the Sound subchapter of the Core API Reference chapter.

2.4 Channels

A channel can be thought of as a single "voice" in your game's mix. Each channel is the source of an audio signal, which it routes into the DSP graph to be mixed.

Provided the sound isn't a stream, you can play multiple channels based on the same sound contemporaneously, without needing to create multiple instances of that sound. Calling `System::playSound` again while one or more channels based on that sound is already playing creates and plays a new channel based on that sound, without stopping the already-playing channels. This allows you to save resources by only keeping one copy of a sound in memory, no matter how many instances of that sound you need to play.

If a sound is a stream (which is to say, if it was created by with `System::createStream` or by calling `System::createSound` with FMOD_CREATESTREAM), calling `System::playSound` while a channel based on that sound is already playing instead causes it to restart. If you want to play multiple instances of a streaming sound's sample data, you can do so by creating multiple sounds based on the same sample data, and playing each one separately.

For more information about channels, see the Channel subchapter of the Core API Reference chapter.

2.4.1 Channel Virtualization

Channels don't necessarily consume resources while they're playing. If a channel falls silent, or if it's the quietest channel when there are too many channels playing, it can be "virtualized." Virtualized channels are abstracted away so that they don't consume resources; if they subsequently become loud enough that they should be audible, or if the number of playing channels drops back down below the limit, they stop being virtual and resume playing as if they had never stopped. Because channels are only virtualized if they're silent or quieter than all other non-virtual channels, this normally has no apparent effect on how the game sounds, but helps save a lot of resources.

For more information about virtualization, see the Virtual Voice System section of the Managing Resources in the Core API chapter.

2.5 Channel Groups

A channel group functions as a container for channels and other channel groups. Each channel group creates a submix of the signals output by the channels and channel groups it contains. This means that you can treat channel groups like buses, routing other channels and channel groups into them in order to better control your mix, and putting DSPs on them to process and modify their submixes.

While it is possible to put DSPs onto channels, it's usually more resource-efficient to put them onto channel groups, as doing so allows a single DSP unit to process the submixed output of multiple channels.

For more information about channel groups, see the ChannelGroup section of the Core API Reference chapter.

2.6 DSP Units

A DSP (or Digital Signal Processor) unit takes PCM audio input and transforms it to create PCM audio output. DSPs are sometimes called effects.

Commonly-used DSPs include the panner, which is used to spatialize and pan signals, and the fader, which is used to adjust volume.

DSPs are usually applied to channel groups in order to modify their submixes in various ways, though they can also be applied to channels. Multiple DSPs can be applied to the same channel group or channel.

For more information about using DSPs, see the Using DSP Effects in the Core API chapter. For more information about specific DSPs, see the Effects Reference chapter.

2.7 FMOD Sound Banks

While the FMOD Core API can play loose audio files in a variety of formats, it can also load and play .fsb files. .fsb files, also known as FMOD sound banks, are a container format optimized for loading and playing sounds in games. You can create FSB files for your game by using the fsbank.exe and fsbankcl.exe applications that comes included with the FMOD Engine.

.fsb files provide a number of benefits over other audio file formats, including:

- **No-seek loading.** FSB loading supports three continuous file reads: A main header read, a sub-sound metadata read, and a raw audio data read.
- **Memory points.** An FSB can be loaded into memory and simply 'pointed to' so that FMOD uses the memory where it is, and does not allocate any extra memory. See FMOD_OPENMEMORY_POINT.
- **Low memory overhead.** A lot of file formats contain fluff such as tags and metadata. FSB stores information in compressed, bit packed formats with minimal overhead for optimum efficiency.
- **Multiple sounds in one file.** Thousands of sounds can be stored inside one .fsb file, and selected by the API function Sound::getSubSound. Because a single .fsb file can include the sample data used by multiple different sounds, .fsb files can also function as loading units for your game's sample data. As loading an .fsb file that contains the sample data for multiple

different sounds requires involves less overhead than loading the sample data for all of those sounds individually, this saves resources in any situation where multiple different sounds' sample data would need to be loaded at the same time.

- **Efficient Ogg Vorbis.** FSB strips out the 'Ogg' and keeps the 'Vorbis'. This means that one codebook can be shared between all sounds, saving megabytes of memory compared to loading .ogg files individually.
- **FADPCM codec support.** The FMOD Engine supports a very efficient ADPCM variant called FADPCM. This variant is many times faster than a standard ADPCM decoder as it does not require branching, and is therefore very efficient on mobile devices. The quality is also far superior than most ADPCM variants, and lacks the 'hiss' notable in those formats.

For more information about FMOD sound banks, see the Supported File Formats section of the Loading and Playing Sounds in the Core API chapter.

2.8 3D Sounds and Channels

A 3D channel or 3D channel group is created with the FMOD_3D flag. This flag allows a channel or channel group to have a defined position, orientation, and velocity in 3D space, set by ChannelControl::set3DAttributes. These 3D attributes can be used by DSPs on that channel or channel group, allowing them to process the signal of that channel or channel group in different ways depending on the 3D attributes' values. The most common reason for making a channel or channel group 3D is to spatialize it with a panner effect, panning and attenuating the signal to make it seem to come from a specific direction and distance.

Sounds can be set to 2D or 3D as well, by specifying the FMOD_3D flag when creating them with System::createSound. In most cases, this isn't strictly necessary - you can make any channel 2D or 3D, regardless of which of those the sound is - but setting a sound to be 2D or 3D makes that the default for all channels based on that sound.

2.9 Streams

Streaming is a way of playing sound without having to load the entire audio asset to be played into memory first. Instead, the sample data is loaded piecemeal into a ring buffer. Each piece of the sample data is loaded into the buffer shortly before it is played and overwritten again as soon as it has finished playing, ensuring that only a tiny amount of the stream's sample data is stored in memory at any given time.

The FMOD Engine supports streaming from file, memory, user callbacks, and http/shoutcast/icecast sources.

Streaming sounds are created using System::createStream or by adding the `FMOD_CREATESTREAM` flag to System::createSound. Each streaming sound plays exactly one instance of its associated sample data. To play multiple instances of the same piece of sample data as streams, you must open and play a stream multiple times. Each such opened stream is its own sound.

Streaming sounds are mostly used for music and dialogue, though they may be used for other sounds as well.

2.10 Audio Devices

An audio device is any piece of hardware that accepts an audio signal and behaves differently according to the signal it receives.

Most audio devices are designed to produce sound: Speakers, headphones, controller speakers, VoIP headsets, and so on. Some audio devices instead use the audio signals they receive for other purposes. For example, the vibrators in some controllers accept audio data and modulate the intensity and frequency of their vibration based on the data they receive.

The Core API is capable of outputting audio signals to audio devices.

FMOD Engine User Manual 2.03.13 (2026-03-31). © 2026 Firelight Technologies Pty Ltd.

© 2026 Firelight Technologies Pty Ltd | Made in Melbourne